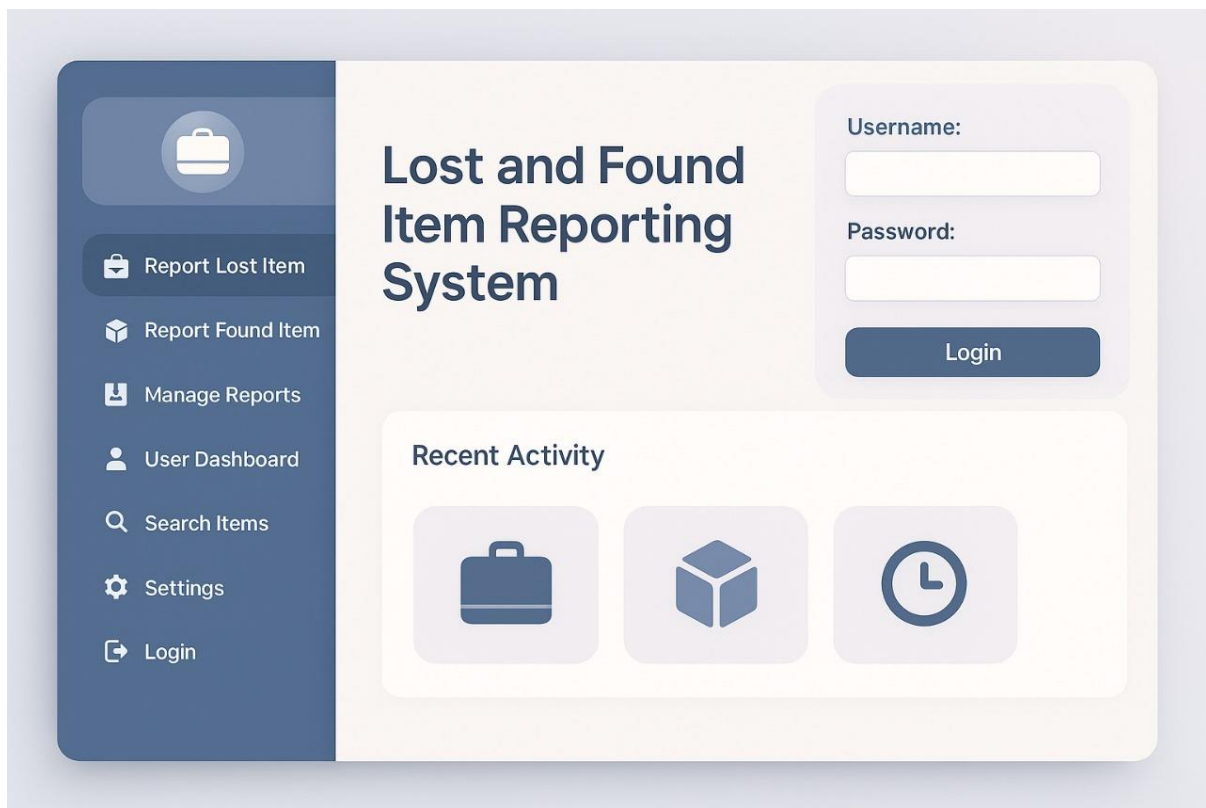


Lost and Found System - Report

Prepared by: Yug Patel



Index

S.No	Title	Page No.
1.	Abstract	Page 1
2.	Introduction	Page 1
3.	Problem Statement	Page 1
4.	Objectives	Page 3
5.	Tools and Technologies Used	Page 3
6.	System Architecture	Page 3
7.	Functional Requirements	Page 4
8.	Non-Functional Requirements	Page 4
9.	System Design	Page 4
10.	Class Design	Page 5
11.	GUI Design	Page 5
12.	Module Description	Page 6
13.	File Handling Strategy	Page 6
14.	Lost Item Reporting Module	Page 6

S.No	Title	Page No.
15.	Found Item Handling Module	Page 7
16.	Status Checking and Updates	Page 7
17.	Viewing Reports by Date	Page 7
18.	User Authentication	Page 7
19.	Error Handling and Validation	Page 8
20.	Test Cases and Output Screens	Page 8
21.	Challenges Faced	Page 8
22.	Images	Page 9
23.	Conclusion	Page 11

1. Abstract

The *Lost and Found Item Reporting System* is a desktop-based application developed using Java to streamline the process of managing lost and found items within institutions. The goal of the application is to replace traditional manual record-keeping with a structured, automated solution that ensures reliability, ease of use, and secure data storage. The system is modular in design and uses AWT (Abstract Window Toolkit) for its graphical interface, making it lightweight and efficient. It is particularly suitable for academic environments, hostels, or office premises where the loss and recovery of personal belongings are a frequent issue.

Key Features:

Java-based application using AWT for GUI.

Modular architecture with separate panels for each function.

File-based data storage using the java.io package.

Supports item reporting, status updates, and complaint tracking.

Designed for educational institutions or small organizations.

2. Introduction

Lost items are a common occurrence in shared environments such as colleges, schools, or workplaces. Often, there is no formalized system in place to report and recover these items, which results in inefficiencies and unclaimed belongings. This project introduces a structured application that provides a digital platform for users to report lost items and for administrative staff to track, update, and verify reports. Through its intuitive user interface and well-organized modules, the system helps maintain a traceable and accountable method for item recovery.

Highlights:

Addresses the inefficiency of verbal or handwritten lost item reports.

Enables digital recording, categorization, and searching of records.

Reduces dependency on physical logs and manual processes.

Promotes accountability by storing data with time and user identity stamps.

Easily extendable for larger institutions in future versions.

3. Problem Statement

The existing approach to handling lost and found items in most institutions is either non-existent or handled through informal means such as announcements, notice boards, or

notebooks. These methods are not only inefficient but also prone to loss of data, miscommunication, and human error. This lack of a system causes delays in recovery and discourages both the reporting and returning of found items. A centralized, automated solution is necessary to ensure items can be tracked, reported, and claimed systematically.

4. Objectives

The main objective of the Lost and Found Item Reporting System is to develop a reliable, user-friendly, and efficient platform for the management of lost and found items in institutional environments. By automating the complaint registration process and ensuring structured data storage, the system enhances traceability, speeds up item recovery, and improves overall user satisfaction. This digital solution is intended not only to solve present inefficiencies but also to support future expansion and system integration.

Detailed Points:

- **Streamlining Complaint Reporting:** The system aims to simplify the way users report lost items. Instead of informal verbal communication, the application allows users to input structured details through a digital form. This ensures that all necessary data—such as roll number, item category, date, and location—is captured accurately.
 - **Maintaining a Clear and Searchable Log:** Each complaint is stored in a standardized format inside a text file categorized by date. This organized data storage allows administrators or users to easily search for complaints using filters such as roll number or date, improving data accessibility and historical tracking.
 - **Facilitating Status Updates:** Once a complaint is submitted, authorized personnel can review the item's details and update its status. This ensures real-time communication between users and the administration and helps track whether an item has been found, returned, or is still missing.
 - **Providing Scalable and User-Friendly Experience:** The design ensures that users with limited technical knowledge can navigate and use the system comfortably. Additionally, the application has been structured with modularity to support future enhancements such as database connectivity, search optimization, or even cloud-based deployment.
-

5. System Architecture

The architecture of the Lost and Found Item Reporting System is designed with a layered and modular approach, allowing for separation of responsibilities and future scalability. At its core, the system is broken into three main layers: the presentation layer, logic layer, and data layer. The **presentation layer** is responsible for all user interactions via GUI components such as text fields, labels, and buttons. It allows users to input or view data in a visually intuitive way. The **logic layer** acts as the brain, interpreting user actions, validating entries, and ensuring correct flow from one operation to another. Lastly, the **data layer** manages persistent storage using plain text files, abstracting all file read-write operations behind encapsulated methods.

A key highlight is the use of CardLayout, a Java layout manager that makes it easy to switch between panels (or screens) without creating multiple windows. This simplifies navigation and creates a seamless single-window interface that reduces system resource usage.

Key Characteristics:

- **Layered separation** ensures better maintainability, testability, and reusability.
 - **Modular components** are loosely coupled to allow independent updates.
 - **CardLayout** allows efficient switching between panels like Login, Dashboard, and Report.
 - **Encapsulation** helps prevent direct manipulation of UI components outside their designated classes.
-

6.Functional Requirements

Functional requirements define the core operations the system must perform to fulfill user needs. This system primarily focuses on facilitating the reporting, tracking, and updating of lost and found item complaints within an institution. From the user's perspective, the system should allow secure login, easy form submission for lost items, and real-time updates of complaint statuses. For administrators or designated staff, it must enable navigation through records and ensure that found items can be matched with the reported complaints.

Data integrity is maintained through strict input validation, and file operations are handled securely to prevent data loss. The system also supports search functionalities based on roll number and date, allowing users to retrieve only the relevant data.

Functional Capabilities:

- **Login and Dashboard Access:** Only authenticated users can access internal functionalities.
 - **Lost Item Reporting:** Collects essential information such as roll number, item category, date, and description.
 - **Complaint Status Updates:** Authorized users can view and update the status of found items.
 - **Search and Retrieval:** Users can query records based on specific dates and roll numbers.
 - **File Persistence:** All submitted data is written into structured text files securely.
-

7.Non-Functional Requirements

While functional requirements ensure the system works correctly, non-functional requirements define *how well* it performs. These include reliability, usability, performance, portability, and scalability. The system is designed to be lightweight and efficient, capable of running smoothly even with limited system resources. The GUI maintains visual uniformity, ensuring all users—regardless of technical ability—can interact with the system easily.

Performance is optimized by minimizing memory usage and reducing unnecessary redraws or reloads of panels. The system is portable as it only requires Java Runtime Environment (JRE) to execute, making it deployable on any platform that supports Java. Future scalability is also considered, with the system architecture allowing for feature additions like email alerts, database integration, or user role management.

Core Aspects:

- **Reliability:** Performs consistently under expected usage conditions.
 - **Responsiveness:** Fast panel switching and low memory usage.
 - **User-Friendly Interface:** Intuitive layout and consistent button/function behavior.
 - **Portability:** Can run on any system with Java installed.
 - **Scalability:** Structure supports addition of new modules without code overhaul.
-

8.System Design

The overall system design follows principles of modularity and component separation. Each major feature (like reporting or updating complaints) is implemented in its own Panel subclass. These panels are controlled and switched dynamically using the CardLayout manager. Event-driven programming is central—each action (button press, field input) is handled via ActionListener, ensuring modular event handling and better maintainability.

GUI components are positioned using **absolute layout**, which, while static, provides precise control over element placement—important for fixed-screen environments such as school or kiosk-based use.

Design Highlights:

- **Panel-based modularity:** Each feature is self-contained in its panel.
 - **CardLayout usage:** Enables a single-window interface with easy transitions.
 - **Event-Driven:** All actions are managed by listeners, keeping logic clean.
 - **Absolute Layout:** Ensures pixel-perfect control over UI for consistent design.
-

9. Class Design

The Java application is structured into several specialized classes, each corresponding to a distinct screen or functional unit. This promotes **object-oriented clarity**, making it easier to track bugs, test components, and update individual parts. For instance, UserPanel handles login-related UI and logic, while ReportLost captures user inputs for lost items. The Dashboard class acts as a navigation hub, directing users to relevant panels based on button selections.

Moreover, class-level encapsulation is observed—each panel or module exposes only the necessary parts and hides internal details, keeping the architecture clean.

Key Classes and Responsibilities:

- **UserPanel:** Login screen management and user authentication.
 - **Dashboard:** Main navigation screen with buttons for accessing other modules.
 - **ReportLost:** Takes user input for lost item complaints.
 - **FoundPanel:** Displays records to search, navigate, and mark found items.
 - **UpdatePanel/UpdateReport:** Fetches and updates complaint statuses.
 - **AllLostItemPanel:** Retrieves all reports submitted on a given date for admin review.
-

10. GUI Design

The graphical user interface (GUI) of the Lost and Found Item Reporting System is designed to offer a clear, clean, and user-friendly experience. The layout is kept minimal and avoids unnecessary graphics or flashy colors to reduce cognitive load. Neutral background tones are paired with blue highlights to draw attention to actionable elements like buttons or input fields. The GUI components—such as labels, text fields, and buttons—are uniformly sized and positioned with ample spacing to avoid clutter.

The application uses **CardLayout**, so all interface panels (login, dashboard, report submission, search, update, and admin view) are part of a single main frame. This avoids opening new windows and improves the fluidity of navigation. Users, especially those unfamiliar with technical systems, benefit from intuitive navigation, simple prompts, and immediate feedback after each action (e.g., confirmation messages, error alerts).

Key Design Elements:

- **Color and Aesthetics:** Neutral backgrounds with blue highlights for clarity and focus.
- **Consistency:** Uniform label and field sizes across all forms.
- **CardLayout Navigation:** Smooth transitions between interface screens.
- **Accessibility:** Labels and buttons are clearly worded to avoid confusion.

- **User Feedback:** On-screen prompts confirm success or warn of errors, improving interaction flow.
-

11. Module Description

Each module in the system is tailored to perform a unique and specific user task. The modular approach enhances both user experience and code maintainability. From the user's point of view, every function—whether it's submitting a lost item, checking complaint status, or viewing all reports—feels like a focused, independent unit. Internally, this is implemented as self-contained panels or classes, each loading only the necessary resources when activated.

Modules are activated through the dashboard and perform their function without interfering with other parts of the system. This modularity also supports future scalability. For example, a new feature like image upload or QR code tagging can be introduced as a new panel without disrupting the rest of the system.

Modules and Roles:

- **Lost Item Reporting Module:** Accepts and stores new lost item complaints from users.
 - **Found Item Handling Module:** Allows navigation and status update of submitted complaints.
 - **Update Module:** Used by authorized users to change the status (e.g., from “not found” to “found”).
 - **View All Module:** Aggregates all complaints on a user-specified date for administrative review.
 - **Login Module:** Ensures restricted access and directs users to the dashboard.
-

12. File Handling Strategy

The system does not use a database; instead, it handles persistent data using text files. This choice keeps the application lightweight and deployable without additional setup. Each file is named according to the date of the report, e.g., `LostItem_2025-07-01.txt`, making it easy to identify, search, and segregate data by time.

When users submit a complaint, the system uses `FileWriter` in append mode to avoid overwriting existing data. Complaints are stored line-by-line, each containing labeled fields such as Roll No, Description, Category, Date, Location, and Status. When records need to be retrieved or updated, they are read using `BufferedReader`. Simple parsing logic is used to extract data from the text format.

Key Features of File Handling:

- **Storage Format:** Text files named by date ensure organized archiving.
 - **Writing:** `FileWriter` (append mode) preserves existing entries while adding new ones.
 - **Reading:** `BufferedReader` is used to load and display complaints in modules.
 - **Data Format:** Entries are stored in labeled key-value pairs to simplify parsing.
 - **Status Flags:** Each complaint includes a status value (e.g., 0 = not found, 1 = found) for easy filtering.
-

13. Lost Item Reporting Module

This module is the entry point for users who wish to report a lost item. It presents a form asking for essential fields such as Roll Number, Item Category, Description, Date, and Location. Upon submission, the system checks that all fields are filled. If any field is empty, an error message is displayed and submission is blocked.

Once validated, the entry is formatted into a structured line and saved into the corresponding text file using the file handling logic discussed earlier. Each submitted

entry includes a **status flag of 0**, which represents that the item is not yet found. This flag plays an important role in later filtering and status updates.

Functional Flow:

- **Data Collection:** Inputs are taken through text fields and dropdowns.
 - **Validation:** Empty field check before submission.
 - **Storage:** Entries saved to text file corresponding to the report date.
 - **Feedback:** Confirmation message displayed upon successful entry.
 - **Initial Status:** Each item marked as "not found" (status = 0).
-

14. Found Item Handling Module

This module is primarily designed for staff or authorized users to browse submitted complaints and identify matches for items found physically on the premises. Users input a **roll number and date**, and the system loads matching complaints from the file corresponding to that date.

The complaints are displayed one at a time in a read-only format, allowing users to navigate using “Next” and “Previous” buttons. If a match is found, the staff member can mark the complaint as “found.” This updates the status flag in the corresponding file (or stores the change in a temporary structure to be overwritten in the file). This module acts as a bridge between submitted data and physical item recovery.

Core Functionality:

- **Search Input:** Roll number and date are required for filtering.
- **Navigation:** Scroll through complaints using “Next” and “Previous.”
- **Matching:** Identify items based on description and mark them as found.
- **Status Update:** Updates flag from 0 (not found) to 1 (found).
- **Controlled Access:** Only staff or admin can perform updates.

Status Checking and Updates

This module allows users to verify the **current status of their previously submitted lost item complaints**. It provides a simple interface where users input their **Roll Number and the date** on which the complaint was submitted. The system then searches the relevant file and displays any matching records.

Each complaint is shown in a clean, read-only format so that users can confirm whether their item has been found or is still pending. For administrative users or authorized personnel, this module also allows **in-place status updates**. Once a match is verified, the status can be toggled from "not found" (status = 0) to "found" (status = 1), helping ensure real-time accuracy of complaint records. Confirmation messages are displayed to ensure users are aware of changes made.

Key Operations:

- **Input Fields:** Roll number and date are used to locate complaints.
 - **Inline Display:** Matching complaints are shown directly within the interface.
 - **Status Indicators:** Clearly displays whether the item is found or not.
 - **Editable by Admin:** Authorized users can update the status directly from this panel.
 - **Feedback System:** Confirmation or error messages are shown based on user action.
-

15. Viewing Reports by Date

This module is designed for **administrators or supervisors** who need to see a daily summary of complaints. By entering a specific date, the system locates the corresponding text file and parses each complaint entry. All retrieved records are shown sequentially in a scrollable area.

This view helps in recognizing **patterns or trends**, such as frequent item loss in a particular location or recurring loss of similar items. This information can assist in

decision-making for preventive measures. It also aids in cross-verification in the case of disputes or inventory checks.

Functional Benefits:

- **Targeted Summary:** Shows all lost items reported on a specific date.
 - **Administrative Use:** Useful for monitoring, audits, or trend analysis.
 - **Easy Navigation:** Records are presented one after another in a single scroll view.
 - **Clear Formatting:** Entries are well-labeled for quick scanning.
 - **Supports Cross-Matching:** Facilitates checking found items against recent reports.
-

16. User Authentication

Authentication is the first step in accessing the system, implemented through a **basic login mechanism**. When the application is launched, users are directed to the login panel where a predefined username and password are required to proceed. This check ensures that only **authorized users** can access internal modules like reporting, updates, and viewing all complaints.

Though basic, the current mechanism prevents unauthorized access in an institutional setting (like schools or colleges). In future upgrades, a more advanced authentication system can be introduced—using encrypted passwords, login logs, and differentiated user roles (e.g., student, admin, security staff).

Current Authentication Model:

- **Login Panel:** Simple form with username and password fields.
 - **Credential Check:** Validates against predefined values hardcoded in the application.
 - **Access Control:** Blocks unauthorized users from using any core features.
 - **Scalability Ready:** Design allows easy upgrade to hashed passwords or file/database-based credential systems.
 - **Post-Login Redirect:** Successful login directs users to the dashboard for further action.
-

17. Error Handling and Validation

A critical aspect of this system is its **robust error handling** and input validation strategy. User inputs are validated to ensure that no field is left empty. Any missing or invalid data results in clear, specific error messages that guide the user to correct the issue before submission. This prevents garbage data from entering the system.

On the backend, file operations and UI logic are protected using try-catch blocks to manage **runtime exceptions**, such as file not found, null values, or I/O errors. When exceptions occur, the system not only logs the problem internally but also informs the user using a dialog or message label, ensuring transparency and ease of troubleshooting.

Protection Mechanisms:

- **Input Validation:** Checks for empty fields or invalid formats before processing.
 - **Try-Catch Blocks:** Used around file and UI logic to catch and handle errors.
 - **User Alerts:** Meaningful error or success messages are displayed in the GUI.
 - **Data Integrity:** Prevents bad or incomplete data from being saved to the system.
 - **System Stability:** Ensures that exceptions do not cause application crashes.
-

18. Test Cases and Output Screens

The system has undergone structured testing using **manual test cases** to ensure its functionality aligns with requirements. Test inputs include both valid and invalid data, edge cases (e.g., extremely long text or duplicate entries), and user behavior simulations (e.g., skipping fields or pressing buttons in wrong sequence).

Screenshots or output text areas were reviewed after each operation to ensure that the system responds with appropriate messages, creates or updates text files accurately, and displays data without delay. Navigational buttons were also tested to confirm that they load the correct panels or records.

Test Case Outcomes:

- **Invalid Input Handling:** Fields left empty trigger user alerts.
- **Valid Input Processing:** Correct entries are stored successfully with confirmation.
- **Status Navigation:** Forward/backward navigation works without overlap or skipping.
- **Data Visibility:** Records are loaded and displayed as expected from text files.
- **Error Simulation:** Missing files or corrupt inputs are managed with clear messages.

19. Challenges Faced

Several challenges were encountered during the design and development of this system. The first major difficulty was implementing a **file-based storage system** that could support structured data retrieval and update without a database. Parsing dynamically structured files while preserving data integrity during read/write operations required careful formatting and logic.

Another challenge was maintaining **GUI consistency** across multiple panels. Ensuring that labels, fonts, alignments, and button behaviors remained uniform involved extra effort in layout planning. Memory optimization was also necessary, as large-scale use could cause performance delays if panels were not managed properly with CardLayout.

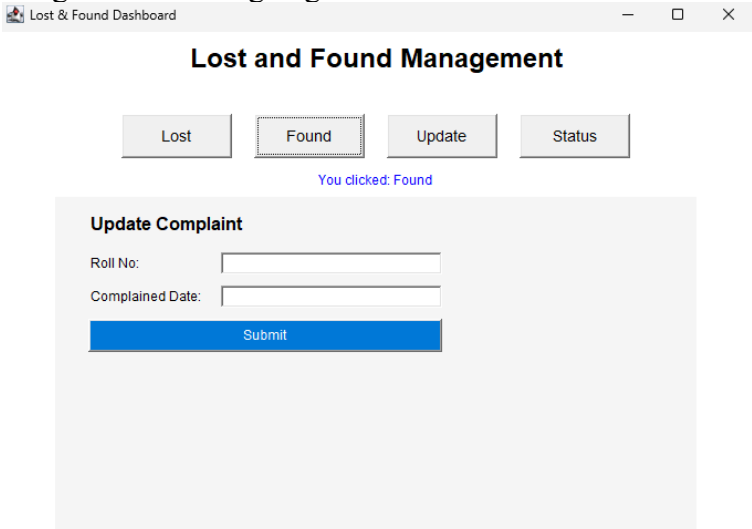
Notable Challenges:

- **File Handling Complexity:** Achieving reliable read/write without a database.
 - **UI Consistency:** Matching layout and interaction behavior across all screens.
 - **Status Flag Logic:** Correctly updating records without overwriting unrelated data.
 - **Navigation Accuracy:** Ensuring complaints load correctly with navigation buttons.
 - **Scalability Consideration:** Designing with future upgrades and feature expansion in mind.
-

20. Images

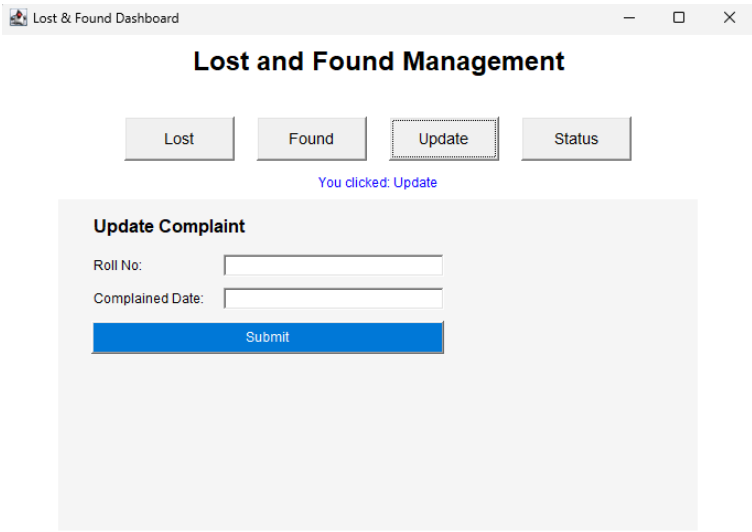
The screenshot displays a web application window titled "Lost & Found Dashboard". The main heading is "Lost and Found Management". Below the heading are four buttons: "Lost", "Found", "Update", and "Status". The "Lost" button is highlighted with a dashed border. Below these buttons is a form titled "Report Lost Item". The form contains five input fields: "Roll No:", "Item category:", "Description:", "Date:", and "Location:". A green "Submit" button is located at the bottom of the form.

Figure 1 : Landing Page



The screenshot shows a web browser window titled "Lost & Found Dashboard". The main heading is "Lost and Found Management". Below the heading are four buttons: "Lost", "Found", "Update", and "Status". The "Found" button is highlighted with a dashed border. Below the buttons, a message reads "You clicked: Found". The main content area is titled "Update Complaint" and contains two input fields: "Roll No:" and "Complained Date:". Below these fields is a blue "Submit" button.

Figure 2: Report Found



The screenshot shows a web browser window titled "Lost & Found Dashboard". The main heading is "Lost and Found Management". Below the heading are four buttons: "Lost", "Found", "Update", and "Status". The "Update" button is highlighted with a dashed border. Below the buttons, a message reads "You clicked: Update". The main content area is titled "Update Complaint" and contains two input fields: "Roll No:" and "Complained Date:". Below these fields is a blue "Submit" button.

Figure 3 : Update Report

The screenshot displays a web application window titled "Lost & Found Dashboard". The main heading is "Lost and Found Management". Below this, there are four buttons: "Lost", "Found", "Update", and "Status". The "Status" button is highlighted, and a message "You clicked: Status" appears below it. The "Complaint Status" section contains two input fields: "Roll No:" with the value "1" and "Complained Date:" with the value "2025-06-23". A blue "Check Status" button is positioned below these fields. A message "Submit button clicked." is displayed at the bottom of the form area.

Figure 4 : Status Report

21. Conclusion : The Lost and Found Item Reporting System demonstrates how basic Java tools can be used to create functional, user-friendly software for real-world scenarios. By leveraging modular design, clean user interfaces, and efficient file handling, the system achieves its goals while laying the groundwork for future improvements and scalability.